オンプレミス構築 教育資料

基礎知識 — サーバー・ネットワーク・Webの仕組み

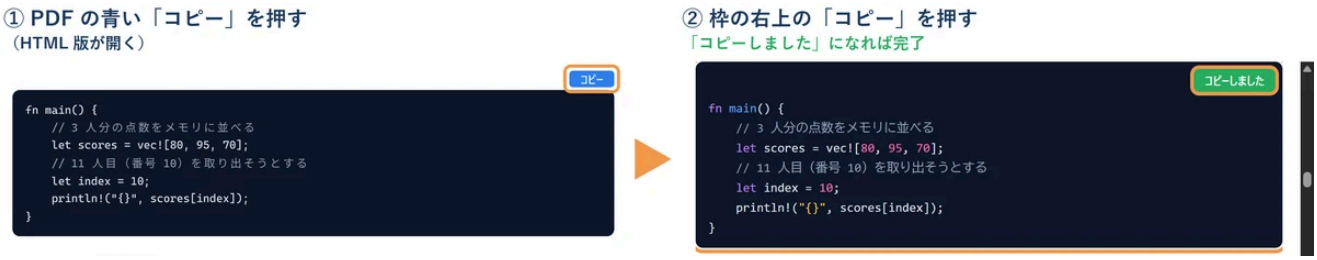
会社の仮想マシンに、社内向けの Web アプリを建てる前に知っておくこと

基礎知識: サーバー・ネットワーク・Webの仕組み

この資料について

この資料は、**インフラ（アプリを動かす土台）の知識がまったく無い人**が、会社のオンプレミスの仮想マシンに Web アプリを建てられるようになるための、最初の一冊です。

- **前提にする環境**: 会社が仮想マシン（VM）を貸してくれて、**OS（Windows Server）だけ**を入れてくれる。それ以外（データベース・Web の入口・アプリ・バックアップなど）は、自分で用意する。
- **この資料の役目**: 構築の手順書（04・05）を読むときに出てくる言葉と考え方を、先にまとめて説明します。手順書の中でも、言葉が出てくるたびに短い説明を入れていますが、「そもそも何のためにあるのか」はこの資料で確かめてください。
- **コマンドやコードのコピー**: PDF のコードの枠の**右上にある青い「コピー」**を押すと、同じ内容の **HTML 版**（`onprem\html` フォルダ）の、そのコードの所が開きます。そこで枠の右上の**「コピー」ボタン**を押すと、字下げや空の行も含めて、そのまま 1 回でコピーできます。コピーしたら、ブラウザーの「戻る」（`Alt` ＋ `←`）で PDF に戻ります。最初から `onprem\html\index.html` を開いて、HTML 版で読んでもかまいません。PDF の文字をマウスでなぞってコピーすると、Chrome や Edge では行の先頭の空白と空の行が消えるので、おすすめしません（PDF を見るソフトの決まった動きで、資料の作り方では直せません）。



- **読み方**: 0 章は「コンピューターの中身」という、いちばん手前の話です。分かっている人は飛ばしてかまいません。図はたとえで描いているので、まず図を見て、それから文を読むと分かりやすいです。

例にするアプリ

この資料と手順書では、次のような**社内向けの業務 Web アプリ**を建てる前提で説明します。

項目	例にするアプリの前提
使う人	社員が、社内のパソコンのブラウザ（Edge・Chrome）で使う。社外からは使わない
できること	依頼を受け付け、承認のお願いをメールで順に回し、進み具合を管理する
保存するもの	依頼の内容・社員データ・承認の記録をデータベースに。図面のデータ（JSON。1 枚 5MB まで・1 件に 10 枚まで）もデータベースに。添付ファイル（JPG・PNG・PDF。1 ファイル 10MB まで）はサーバーのフォルダに
メール	会社のメール（Microsoft 365）を通して、社員の Outlook に届ける
自動の仕事	毎朝 8:00 に、期限の近い依頼や止まっている承認を調べてメールを送る（アプリの中の定期処理）
ログイン	60 分操作が無いと、自動でログアウトする
控え	毎晩、データベースと添付ファイルを控え（バックアップ）に取り、14 日分を残す
環境	同じサーバーに、本番（社員が使う）と検証（試す）を同居させる
通信	社内では HTTPS（暗号化した通信）を使えないので、**HTTP** で動かす（6 章）

資料の一覧

資料	内容
00（この資料）	コンピューター・サーバー・ネットワーク・Web・データベース・メール・運用・セキュリティの基礎知識
01 Windows Server の使い方	サーバーへのログイン（リモート デスクトップ）、PowerShell、サービス・イベント ビューアー・タスク スケジューラなどの管理の道具
02 SQL と PostgreSQL の使い方	データベースへのログイン、SQL の書き方（サンプル付き）、ユーザーと権限、バックアップ
03 技術スタックの比較と選定理由	OS・Web サーバー・データベース・アプリの作り方などの選択肢と、長所・短所、今回の選び方
04 構築手順（Rust＋Topcoat版）	アプリを Rust で作った1つの実行ファイルにして、仮想マシンに建てる手順
05 構築手順（Django＋React版）	同じアプリを Django（Python）と React（JavaScript）で作った場合の手順。React の画面の成果物は Django のプロジェクトの中に置き、Django が同じサーバーで配信する
06 トラブルシューティング	困ったときの調べ方と、よく起きるトラブルの症状・原因・直し方
07 付録	情報システム部門への依頼シート、作業と点検のチェック表
08 付録: よく使うコマンドの早見表	毎日の確認・サービス・通信・ログ・ディスク・PostgreSQL・更新と切り戻しで打つコマンドを、やりたいこと別にまとめた表（実際に打った結果付き）
参考 HTTPS にする場合	HTTPS と証明書の考え方と、あとから HTTPS にするときの手順（今回は使わない）

まだできていない資料は、README の一覧に「作成中」と書いてあります。

> **用語:** インフラ（infrastructure：アプリを動かすための土台の全部。サーバー・OS・ネットワーク・データベース・バックアップなど。電気・水道・道路のような「無いと何も動かない物」と同じ意味で使う）

> **用語:** オンプレミス（on-premises：会社の建物の中や、会社が管理する設備の上でシステムを動かすこと。インターネット上の借り物（クラウド）を使わない。略して「オンプレ」）

0. はじめの一歩: コンピューターの中身

サーバーも、社員のパソコンも、中身は同じ「コンピューター」です。この章では、あとの章を読むのに要る、いちばん手前の話をします。

0.1 コンピューターの4つの部品



図0-1: コンピューターの4つの部品を、事務所の仕事にたとえると

部品	たとえ	役目	足りないと、どうなるか
CPU	考える係	計算や判断をする。数（コア数）が多いほど、同時にたくさんの仕事ができる	画面の表示が遅くなる。たくさんの人が同時に使うと待たされる
メモリ（RAM）	作業机	今使っているプログラムとデータを広げておく。とても速いが、**電源を切ると消える**	机に乗らない分を、ディスクに置き直しながら仕事をするので、急に遅くなる
ディスク（ストレージ）	本棚	ファイルやデータベースを、**電源を切っても残す**。メモリより遅い	満杯になると、データベースもログも書けなくなり、アプリが止まる
ネットワーク（NIC）	電話	ほかの機械と話す	ほかの機械とつながらない

サーバーを借りるときに「vCPU 4・メモリ 16GB・ディスク 200GB」のように頼むのは、この4つのうちの3つの量を決めているのです（04 の 2 章）。

> **用語:** CPU（シーピーユー。Central Processing Unit：コンピューターの頭脳。命令を読んで計算する部品）

> **用語:** コア（CPU の中にある、計算をする単位。4 コアなら、4つの仕事を同時に進められる）

> **用語:** メモリ（RAM。ラム：プログラムが動いている間だけ、データを置いておく場所。速いが、電源を切ると消える）

> **用語:** ディスク（ストレージ：データを長く残しておく場所。SSD（速い）や HDD（安くて大きい）がある）

0.2 データの大きさの単位

データの大きさは、**バイト**（B）という単位で数えます。

単位	大きさ	身近な例
1 バイト（B）	英数字1文字	`A` の1文字
3 バイト	日本語1文字（UTF-8 のとき。0.4）	`依` の1文字
1 KB（キロバイト）	約 1,000 バイト	短いメール1通
1 MB（メガバイト）	約 1,000 KB	スマートフォンの写真1枚が 2〜5MB
1 GB（ギガバイト）	約 1,000 MB	映画1本が数 GB
1 TB（テラバイト）	約 1,000 GB	大きなディスク1台

> **細かい話:** Windows の画面では、1 KB を 1,024 バイトとして数えます。このため、「200GB のディスク」が、画面では 186GB くらいに見えます。故障ではありません。

ディスクの量の見積もり（例）：添付ファイルが1日に 20 件 × 平均 2MB なら、1日 40MB。営業日が年に 250 日なら、1年で 40MB × 250 = 10,000MB ＝ 約 10GB。バックアップを 14 日分残すなら、その分も足します。こうして、「何年分を入れるのに、どれだけのディスクが要るか」を計算します。

通信の速さは、**1秒に何ビット送れるか**（bps）で表します。1 バイト ＝ 8 ビットなので、1Gbps の回線は、理屈の上では1秒に約 125MB を送れます（実際には、その半分くらいと考えておきます）。

> **用語:** ビット（bit：0 か 1 かの、いちばん小さなデータの単位。8 ビットで 1 バイト）

0.3 ファイル・フォルダ・パス

- **ファイル**は、データのひとかたまりです。名前の最後の `.` から後ろを**拡張子**と呼び、ファイルの種類を表します。

拡張子	種類	この資料での例
`.exe`	実行ファイル（ダブルクリックやコマンドで動くプログラム）	`webapp.exe`、`postgresql-18.6-windows-x64.exe`
`.ps1`	PowerShell のスクリプト（コマンドを並べて書いたファイル）	`backup.ps1`
`.msi`	Windows のインストーラー	`rewrite_amd64_en-US.msi`
`.zip`	いくつものファイルを1つにまとめて縮めた物	`webapp-1.0.0.zip`
`.log`	ログ（記録）のファイル	`webapp.out.log`
`.xml`・`.config`・`.toml`・`.conf`	設定ファイル	`webapp-prod.xml`、`web.config`、`postgresql.conf`
`.csv`	表のデータを、カンマで区切って書いた文字のファイル	社員データの取り込み

- **フォルダ**は、ファイルを入れる箱です。フォルダの中にフォルダを作れます。
- ファイルの場所を、ドライブから順に `\`（円記号。英語の画面ではバックスラッシュ）でつないで書いたものを、**パス**と呼びます。

コピー

```
D:\webapp\prod\logs\webapp.out.log
│  │      │    │    └ ファイル名（webapp.out.log）
│  │      │    └ フォルダ（logs）
│  │      └ フォルダ（prod）
│  └ フォルダ（webapp）
└ ドライブ（D ドライブ）
```

- パスに**空白**が入っているときは、コマンドで使うときに `"`（ダブルクォート）で囲みます。囲まないと、空白の所で別の言葉だと思われます。

コピー

```
# 空白が入っているので、" で囲む。先頭の & は「このファイルを実行して」という意味
& "C:\Program Files\PostgreSQL\18\bin\psql.exe" --version
```

> **最初にしておくこと:** エクスプローラーの「表示」メニューで、「**ファイル名拡張子**」と「**隠しファイル**」を表示する設定にします。拡張子が見えないと、`backup.ps1.txt` のような打ち間違いに気付けません。

0.4 文字コード（文字化けの正体）

コンピューターは、文字を**番号**で覚えています。どの文字を何番にするかの決まりを、**文字コード**と呼びます。

文字コード	どこで使われるか	気を付けること
UTF-8	Web、Rust・Python・PostgreSQL の標準。世界中の文字を表せる	今回は、基本的にすべてこれにそろえる
Shift_JIS（CP932）	古い Windows のソフト、Excel の「CSV（コンマ区切り）」で保存したファイル	UTF-8 だと思って読むと、**文字化けする**

文字化けは、「書いたときの決まり」と「読むときの決まり」が違うと起きます。たとえば、Excel で「CSV（コンマ区切り）」として保存した社員データを、アプリが UTF-8 として読むと、日本語が `◆` や意味の無い漢字に化けます。Excel では「**CSV UTF-8（コンマ区切り）**」を選んで保存します。

コピー

```
# ファイルを UTF-8 として読んで、先頭の 5 行を表示する
Get-Content D:\webapp\tools\users.csv -Encoding UTF8 -TotalCount 5
```

確かめること: 日本語の名前が、正しく読めること。化けていたら、ファイルが Shift_JIS で保存されています。

- UTF-8 には、ファイルの先頭に「これは UTF-8 です」という印（**BOM**）を付ける形と、付けない形があります。Windows に最初から入っている PowerShell（5.1）は、**BOM の無い UTF-8 のスクリプトを Shift_JIS だと思って読む**ので、日本語を含むスクリプトは「UTF-8 (BOM 付き)」で保存します（01 Windows Server の使い方 3.7）。
- 行の終わりの印（**改行コード**）にも、Windows 式（CRLF）と Linux 式（LF）の2つがあります。設定ファイルを Windows の「メモ帳」で書く分には、気にしなくてかまいません。

> **用語:** 文字コード（文字と番号の対応の決まり。UTF-8・Shift_JIS など）

> **用語:** 文字化け（文字コードの食い違いで、文字が意味の無い記号に変わって見えること）

0.5 プログラム・コマンド・スクリプト

- **プログラム**は、コンピューターへの命令を並べた物です。Windows では、`.exe` のファイルが、すぐ動かせるプログラムです。
- **コマンド**は、プログラムを、文字で打って動かす命令です。黒い（または青い）画面（**ターミナル**）に打って、Enter キーで実行します。
- **スクリプト**は、コマンドを並べてファイルに保存した物です。同じ作業を、何度でも同じように繰り返せます。

この資料と手順書のコマンドは、Windows の **PowerShell** で打ちます。次のように書いてある所は、1行ずつ打つ（または貼り付ける）ことを表します。`#` から後ろは説明で、打たなくてもかまいません。

コピー

```
# この機械の名前を表示する
hostname

# 今ログインしている自分のアカウントを表示する
whoami

# ドライブごとの、使っている量と空いている量を表示する（Used・Free の列）
Get-PSDrive -PSProvider FileSystem
```

確かめること: `hostname` で、頼んだサーバーの名前（例：`WEBAPP-SV01`）が出ること。`Get-PSDrive` の `Free` で、D ドライブの空きが、頼んだ量に近いこと。

PowerShell の使い方は、**01 Windows Server の使い方**で詳しく説明します。

> **用語:** ターミナル（文字でコンピューターを操作する画面。Windows では「ターミナル」アプリの中で PowerShell を動かす）

> **用語:** スクリプト（コマンドを並べて保存したファイル。PowerShell では拡張子 `.ps1`）

1. 全体像: 社内に「お店」を開く

Web アプリを社内に建てるのは、**社内に1軒のお店を開く**のに似ています。

- **建物**（仮想マシン）は、会社が貸してくれます。建物の中の**内装の基本**（OS）も、会社がやってくれます。
- 自分でするのは、**受付カウンター**（Web の入口。IIS）、**店員**（アプリ）、**帳簿の棚**（データベース）、**郵便を出す係**（メール送信）を用意して、**鍵**（ファイアウォール・アクセス許可）をかけ、**帳簿の控え**（バックアップ）を毎晩取ることです。
- お客さん（社員）は、社内の道（社内ネットワーク）を通って、お店の**住所**（URL）に来ます。お店は、社内の人しか入れない場所にあります。

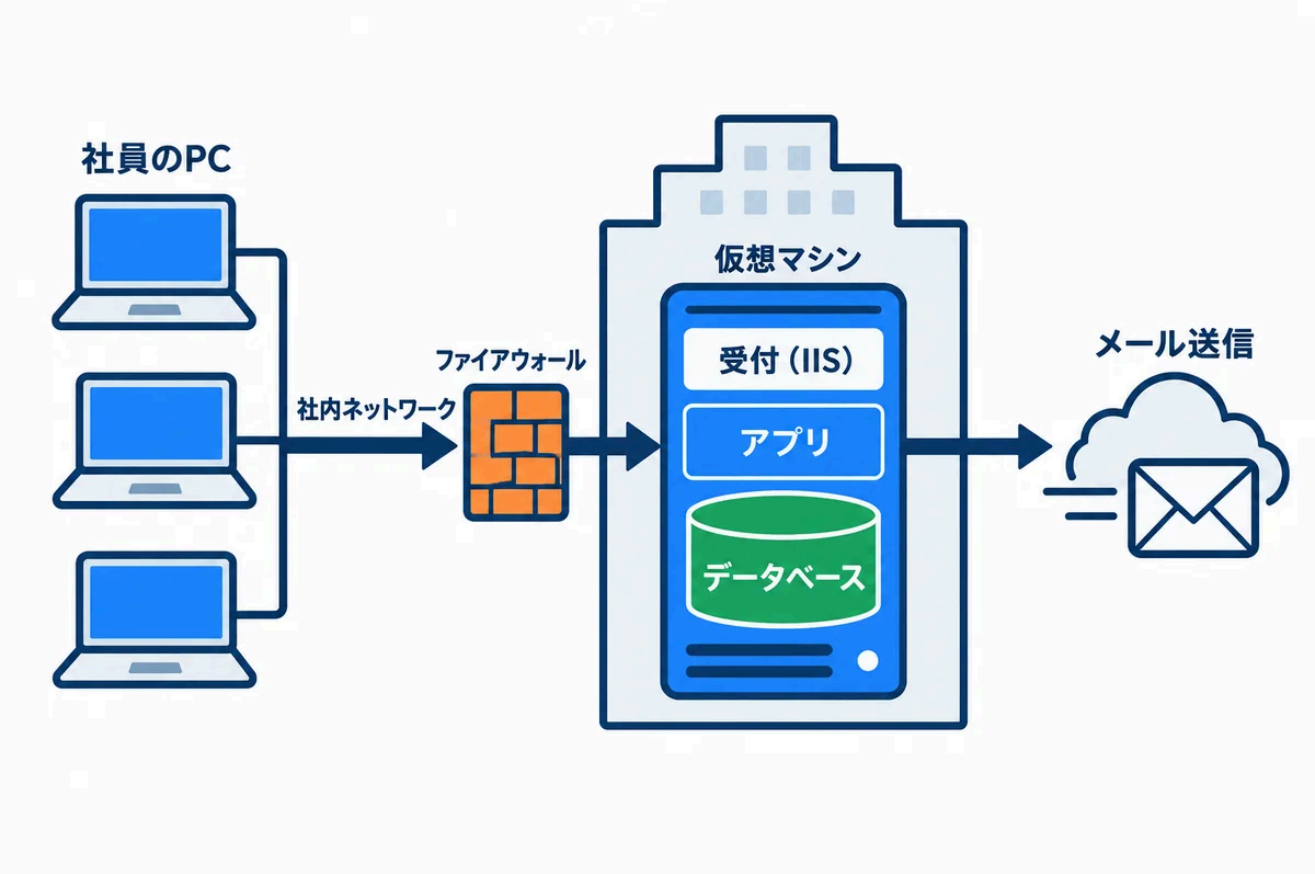


図1-1: 全体像。仮想マシンの中に、受付・アプリ・データベースを自分で用意する

	だれがするか	中身
会社（情報システム部門）がしてくれること	情報システム部門	仮想マシンを貸す・OS を入れる・社内の道（ネットワーク）・DNS への名前の登録・ネットワークの関所・Microsoft 365 のメールのコネクタ
自分がすること	自分	それ以外の全部（データベース・アプリ・受付（IIS）・サーバーの関所・メールの送り方・毎晩の控え・点検）

この資料の以降の章は、この図の部品を1つずつ説明します。

図の部品	たとえ	本当の名前	この資料の章
建物	会社が貸してくれる	仮想マシン（VM）	2
内装の基本	会社が入れてくれる	OS（Windows Server）	3
社内の道・住所	社員がお店に来る道	ネットワーク・IP アドレス・DNS・ポート	4
受付・店員	注文を受けて仕事をする	Web サーバー（IIS）・アプリ	5
封筒（今回は封をしない）	中身を見られないようにする仕組み	HTTP と HTTPS	6
帳簿の棚	データを保管する	データベース（PostgreSQL）	7
郵便局	メールを届ける	SMTP・Microsoft 365	8
毎晩の控え・点検	何かあっても戻せる	バックアップ・ログ・監視	9
鍵・見張り	悪い人を入れない	セキュリティ	10

2. サーバーと仮想マシン

2.1 サーバーとは

サーバーは、ほかのコンピューターからの「お願い」を受けて、仕事をして返すコンピューター（またはプログラム）です。社員のパソコンのブラウザが「依頼の一覧を見せて」とお願いすると、サーバーが一覧を作って返します。



図2-1: クライアントがお願いし、サーバーが返事をする

- お願いをする側を**クライアント**、受ける側を**サーバー**と呼びます。ブラウザはクライアントです。
- サーバーは、たくさんの人のお願いを、いつでも受けられるように、**電源を入れっぱなしで、画面の前に人がいない**状態で動き続けます。だから、「人がログインしていなくても、自動で動き出す」仕組み（サービス。3章）が要ります。

> **用語**: サーバー（server：お願いを受けて仕事をするコンピューターやプログラム。「サービスを提供する（serve）もの」の意味）

> **用語**: クライアント（client：サーバーにお願いをする側。ブラウザや、社員のパソコン）

2.2 仮想マシン（VM）とは

昔は、サーバー1台につき、本物の機械を1台用意していました。今は、1台の大きな機械の中に、**何台ものコンピューターがあるように見せる**技術（仮想化）を使うのがふつうです。

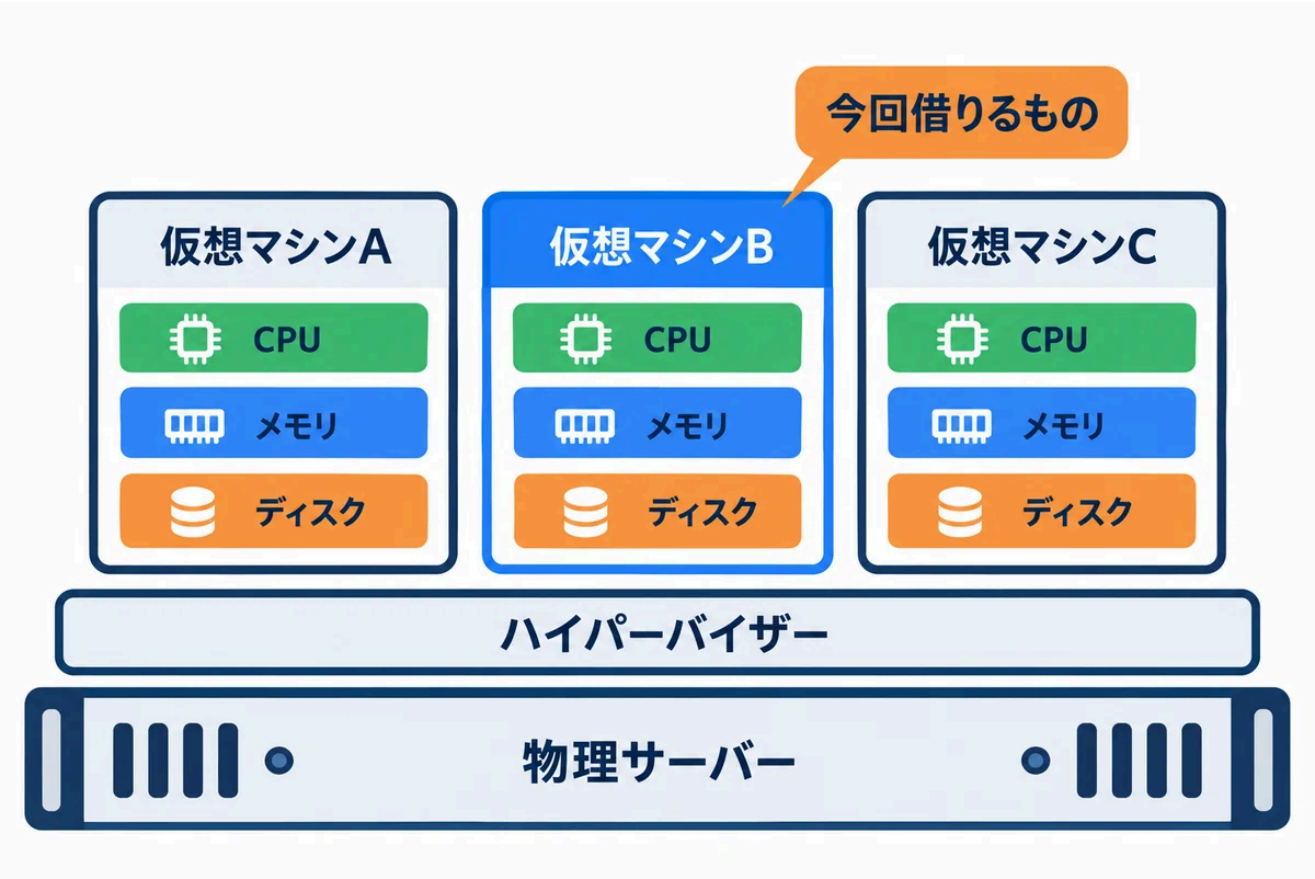


図2-2: 1台の物理サーバーの中に、何台もの仮想マシンを作る

- 本物の機械を**物理サーバー**（ホスト）、その中に作った仮のコンピューターを**仮想マシン**（VM、ゲスト）と呼びます。
- 仮想マシンには、**CPU の数**（**vCPU**）**・メモリ・ディスク**が割り当てられます。足りなくなったら、情報システム部門に頼んで増やしてもらえることが多いです（再起動が要ることがあります）。
- 仮想化の仕組みを動かすソフトを**ハイパーバイザー**と呼びます。会社では VMware vSphere や Microsoft Hyper-V がよく使われます。どれを使っているかは、情報システム部門に聞きます。

> **用語:** 仮想化（virtualization：1台の機械を、何台もの独立したコンピューターのように分けて使う技術）

> **用語:** vCPU（ブイシーピーユー：仮想マシンに割り当てられた CPU の数。仮想 CPU）

> **用語:** スナップショット（snapshot：仮想マシンの「その瞬間の状態」を丸ごと保存しておく機能。作業の前に取っておけば、失敗したときにその瞬間まで戻せる。ただし、**長く残しておくとディスクを圧迫し、遅くなる**ので、作業が終わったら消すのがふつう。**バックアップの代わりにはならない**（9章））

2.3 オンプレミスとクラウドの違い

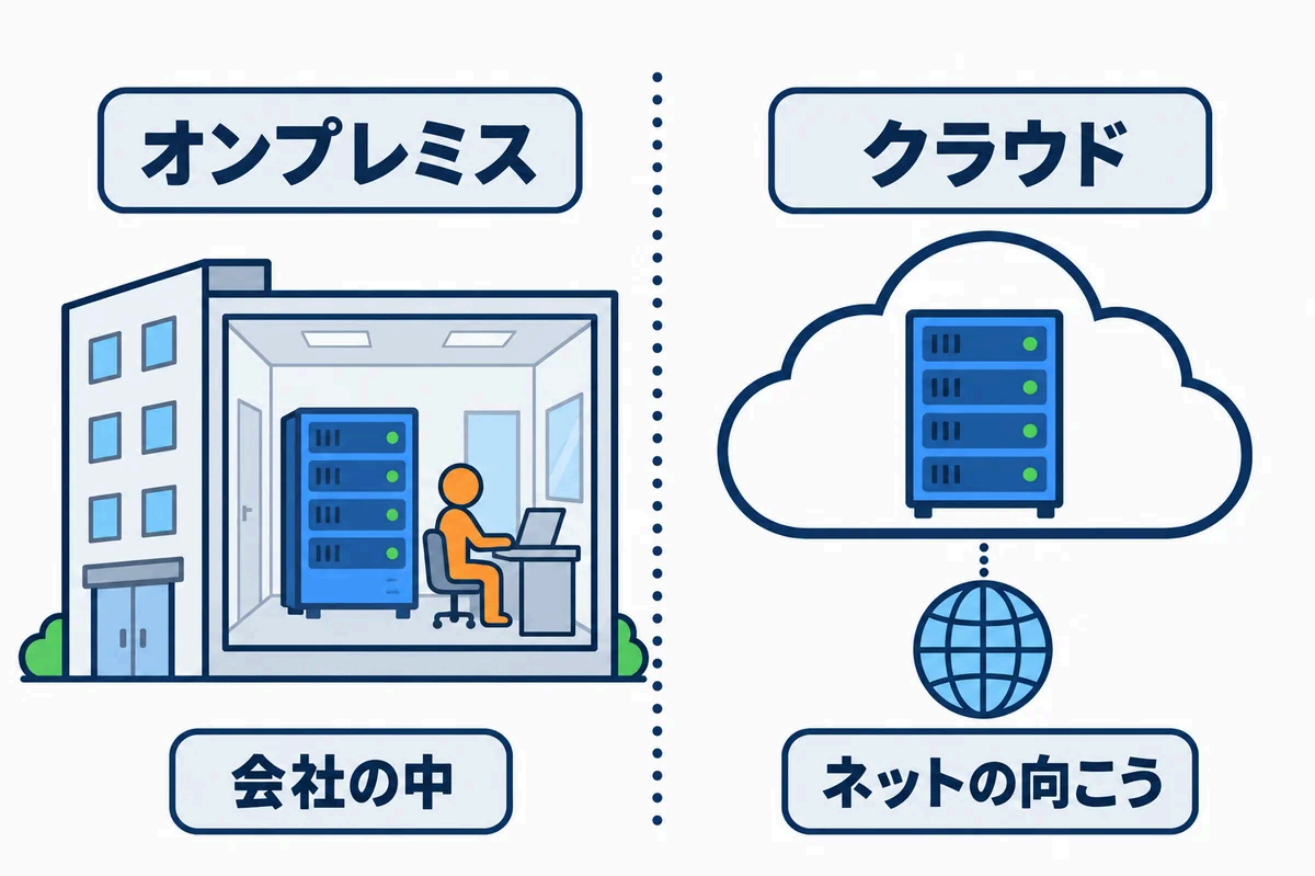


図2-3: オンプレミスとクラウド

	オンプレミス（今回）	**クラウド（Azure・AWS など）**
機械はどこにあるか	会社の中	インターネットの向こうの会社のデータセンター
使える人	社内ネットワークの人だけ（にできる）	設定しだいで、どこからでも
自分でする範囲	今回は OS より上の全部	サービスによって、OS もしなくてよいことがある
インターネット	社内のサーバーは、インターネットにつながらないことが多い	ふつうはつながる

オンプレミスのサーバーは、インターネットにつながらない（または、決まった相手にしかつながらない）ことが多い、という点が大事です。この資料でも、サーバーからインターネットへは「Microsoft 365 へのメール送信（25 番ポート）だけが許される」前提で考えます。

そのため、サーバーの中から「インターネットで最新のソフトを取ってくる」ことはできない前提にします。**インストーラーや部品は、インターネットにつながる開発用のパソコンで入手して、サーバーへ持ち込みます**（手順書 01・02 で、毎回そうします）。

3. OS（Windows Server）の基礎

3.1 OS とは

OS（オペレーティングシステム）は、コンピューターの基本のソフトです。ファイルの保存、プログラムの起動、ネットワークの通信、ユーザーの管理などを受け持ちます。

Windows Server は、サーバー用の Windows です。見た目は社員のパソコンの Windows に似ていますが、サーバーとして長く安定して動かすための機能（IIS、役割と機能の追加、リモート管理など）が入っています。版（2022・2025 など）によって細かい所が違うので、受け取ったら版を確かめます（手順書の 3 章）。

> **用語:** Windows Server の「デスクトップ エクスペリエンス」と「Server Core」（画面（デスクトップ）がある形と、画面が無くコマンドだけで操作する形。初めての人には、デスクトップ エクスペリエンスの方が分かりやすい。どちらで入れてもらうかを、最初に情報システム部門と決める）

3.2 アカウントと権限



図3-1: アカウントごとに、渡す鍵（権限）を変える

- **アカウント**は、コンピューターを使う人（やプログラム）の名前と鍵です。
- **ローカルアカウント**はその1台だけで使えるアカウント、**ドメインアカウント**は会社全体（Active Directory）で使えるアカウントです。会社のサーバーは、ふつうドメインに参加していて、自分の会社のアカウントでログインします。

- **管理者**（Administrators）は、何でもできる強い権限です。インストールや設定には要りますが、**アプリを動かすときには、管理者の権限は使いません。**アプリ専用の、必要なことだけができるアカウント（サービス用アカウント）を使います。これを**最小権限**の考え方と言います（10章）。

> **用語:** Active Directory（アクティブ ディレクトリ：会社のパソコンとユーザーをまとめて管理する Windows の仕組み。略して AD）

> **用語:** 権限（その人やプログラムができることの範囲。ファイルを読む・書く・消す、設定を変える、など）

3.3 プロセスとサービス

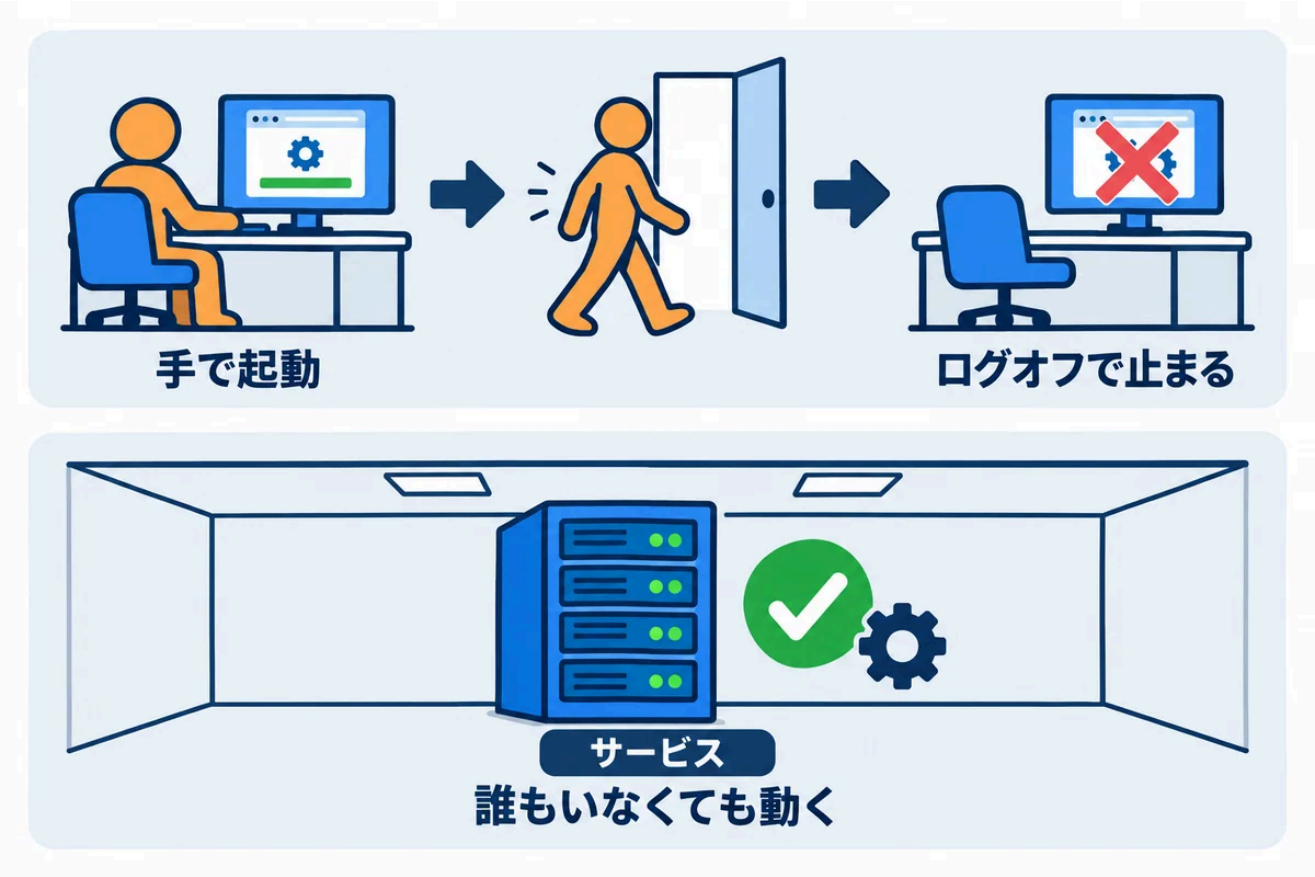


図3-2: 手で起動したプログラムと、サービスの違い

- プログラムを起動すると、OS の中で**プロセス**（動いているプログラム）になります。
- 人がログインしてプログラムを起動すると、その人がログオフしたときにプロセスも終わってしまいます。サーバーでは困るので、**サービス**という形で動かします。サービスは、**Windows の起動と一緒に、誰もログインしていなくても動き出し**、止まったときに自動でもう一度起動させることもできます。
- サービスの一覧は、「サービス」の画面（`services.msc`）か、PowerShell の `Get-Service` で見られます。

コピー

```
# 名前に postgres を含むサービスの、状態（Status）と起動の種類（StartType）を見る
Get-Service -Name *postgres* | Select-Object Name, Status, StartType
```

確かめること: `Status` が `Running`（動いている）、`StartType` が `Automatic`（Windows と一緒に起動）になっていること。サービスの詳しい扱い方は、01 Windows Server の使い方の「サービス」の章で説明します。

> **用語:** サービス（Windows サービス：画面の前に人がいなくても、OS が裏で動かし続けるプログラム。IIS も PostgreSQL も、サービスとして動く）

3.4 ファイル・フォルダとアクセス許可

- Windows のディスクは **NTFS** という形式で、フォルダやファイルごとに「誰が読めて、誰が書けるか」（アクセス許可）を決められます。
- 秘密の設定（データベースのパスワードなど）を書いたファイルは、**管理者とアプリ用のアカウントだけが読める**ようにします。
- サーバーには、OS 用の **C ドライブ**と、データ用の **D ドライブ**（追加のディスク）を用意してもらうことが多いです。アプリのデータやバックアップは、D ドライブに置くと、OS の入れ直しやディスクの拡張がしやすくなります。

3.5 PowerShell

PowerShell は、Windows を文字のコマンドで操作する道具です。画面のクリックでできることの多くは、PowerShell でもできます。手順書は、**同じ結果を何度でも出せるように**、なるべく PowerShell のコマンドで書いています。

- 「管理者として実行」した PowerShell でないと、インストールや設定の変更はできません（スタートボタンを右クリック→「ターミナル（管理者）」など）。
- `#` から後ろはコメント（説明）で、打たなくても何も変わりません。

> **用語:** コマンド（コンピューターへの命令を、文字で打ったもの。打って Enter キーを押すと実行される）

> **用語:** 環境変数（プログラムに「設定」を渡すための、名前と値の組。例： `PORT=8080` 。設定ファイルの代わりに使える）

4. ネットワークの基礎

4.1 IP アドレスとホスト名

- ネットワークにつながる機械には、**IP アドレス**という番号の住所が付いています。例： `10.20.30.40` （IPv4 の形。点で区切った4つの数）。

- サーバーの IP アドレスは、変わると困るので、**固定**にしてもらいます（固定 IP）。
- 番号は覚えにくいので、**ホスト名**（例：`webapp-sv01`）や、会社のドメインまで付けた**完全な名前**（FQDN。例：`webapp-sv01.example.local`）を付けます。
- 名前から IP アドレスを調べる仕組みが **DNS** です。社員が `http://webapp.example.local/` と打つと、DNS が「それは 10.20.30.40 です」と答え、ブラウザはその番号の機械につなぎます。**DNS に名前を登録するのは、ふつう情報システム部門の仕事**です。

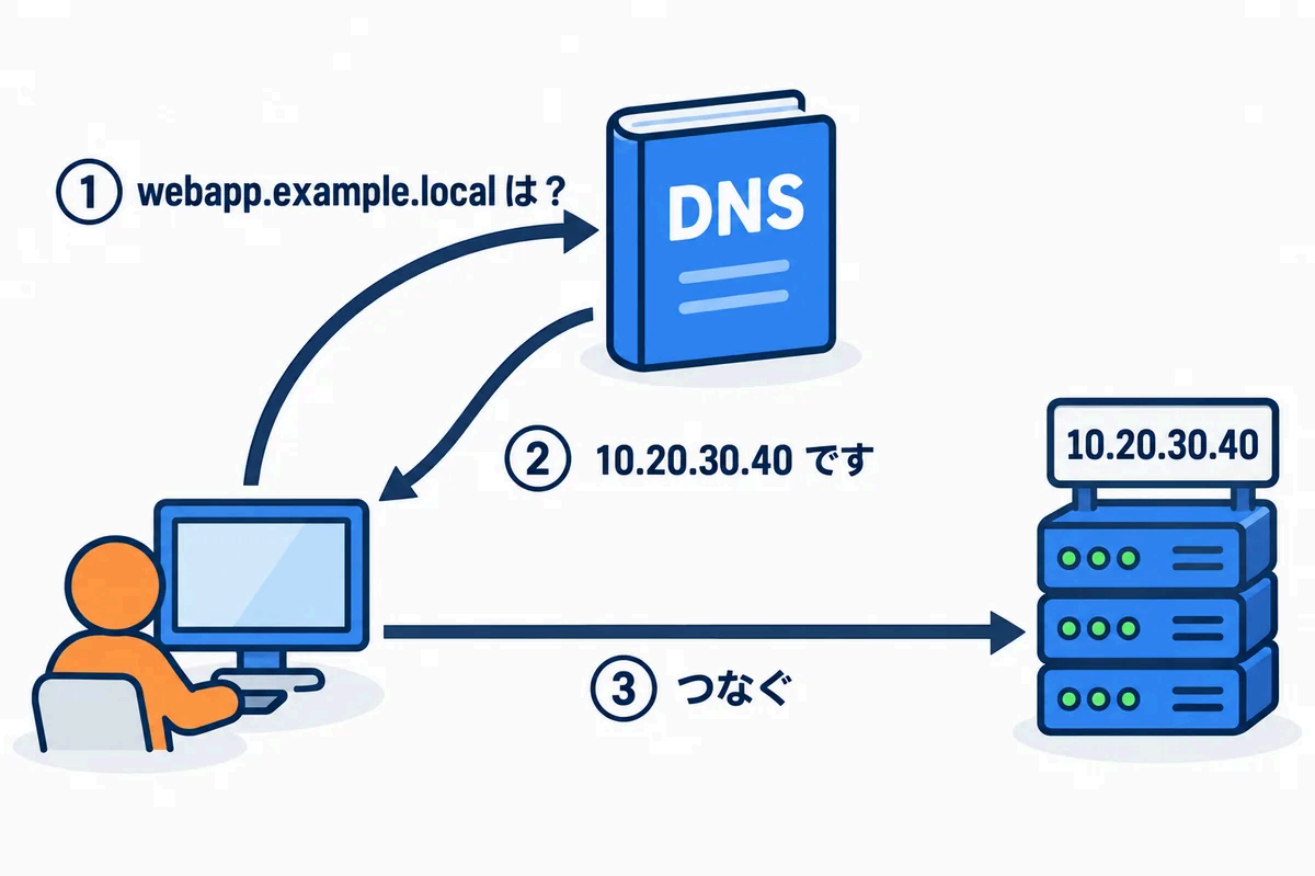


図4-1: DNS は、名前を番号の住所に変える電話帳

> **用語:** IP アドレス（Internet Protocol address：ネットワーク上の機械の番号の住所）

> **用語:** DNS（ディーエヌエス。Domain Name System：名前（例：webapp.example.local）と IP アドレスを結び付ける、ネットワークの電話帳）

> **用語:** FQDN（エフキューディーエヌ。Fully Qualified Domain Name：ホスト名にドメイン名まで付けた、完全な名前）

> **用語:** サブネットマスク・デフォルトゲートウェイ（同じネットワークの範囲を表す数と、ほかのネットワークへ出るときの出口の機械。ふつうは情報システム部門が決めて設定してくれる）

4.2 ポート番号

1台のサーバーの中では、いくつものプログラムが通信を待っています。どのプログラムに届けるかを決める番号が**ポート番号**です。IP アドレスが「建物の住所」なら、ポート番号は「部屋番号」です。



図4-2: IP アドレスは建物の住所、ポート番号は部屋番号

ポート	決まった使い方	今回の使い道
80	HTTP（Web）	社員がアプリを開く入口（本番も検証も。名前で分ける。下の説明）
443	HTTPS（暗号化した Web）	今回は使わない（参考資料）
3389	RDP（リモート デスクトップ）	管理者がサーバーに入る
5432	PostgreSQL	データベース（サーバーの中だけで使う）
25	SMTP（メールの配達）	Microsoft 365 へメールを渡す（外向き）
8080・8081 など	決まりは無い	アプリ本体（サーバーの中だけで使う）

- 社内から受け付けるのは **80 番だけ**にします。本番と検証は、**同じ 80 番で、名前を変えて**分けます（例：本番 `webapp.example.local`、検証 `webapp-stg.example.local`）。2つの名前を、DNS で同じ IP アドレスに向け、受付（IIS）が「どの名前で来たか」を見て、本番と検証に振り分けます。ブラウザは、お願いのたびに「どの名前宛てか」を **Host ヘッダー**に書いて送るので、受付はそれを見て分けられます。

- **localhost**（`127.0.0.1`）は「この機械自身」という特別な住所です。アプリやデータベースを `127.0.0.1` で待ち受けさせると、**同じサーバーの中からしか**つなげません。外から直接さわられないので、安全です。アプリ本体とデータベースは、この形にして、外からは受付（IIS）だけを通します。

> **用語:** ポート番号（port：1台の機械の中で、どのプログラム宛ての通信かを表す番号。0〜65535）

> **用語:** Host ヘッダー（ホスト ヘッダー：ブラウザがお願いに付けて送る「どの名前のサイト宛てか」の情報。1つの IP アドレスとポートで、名前ごとに別のサイトを出すのに使う）

> **用語:** 待ち受け（listen：プログラムが、決まったポートで、通信が来るのを待っていること）

> **用語:** TCP（ティーシーピー：届いたかを確かめながら、順番どおりにデータを送る通信の決まり。Web・メール・データベースは、どれも TCP を使う）

> **用語:** プロトコル（protocol：通信の決まりごと。HTTP・HTTPS・SMTP・RDP など）

4.3 ファイアウォール

ファイアウォールは、通信の**関所**です。決めた通信だけを通し、それ以外は止めます。

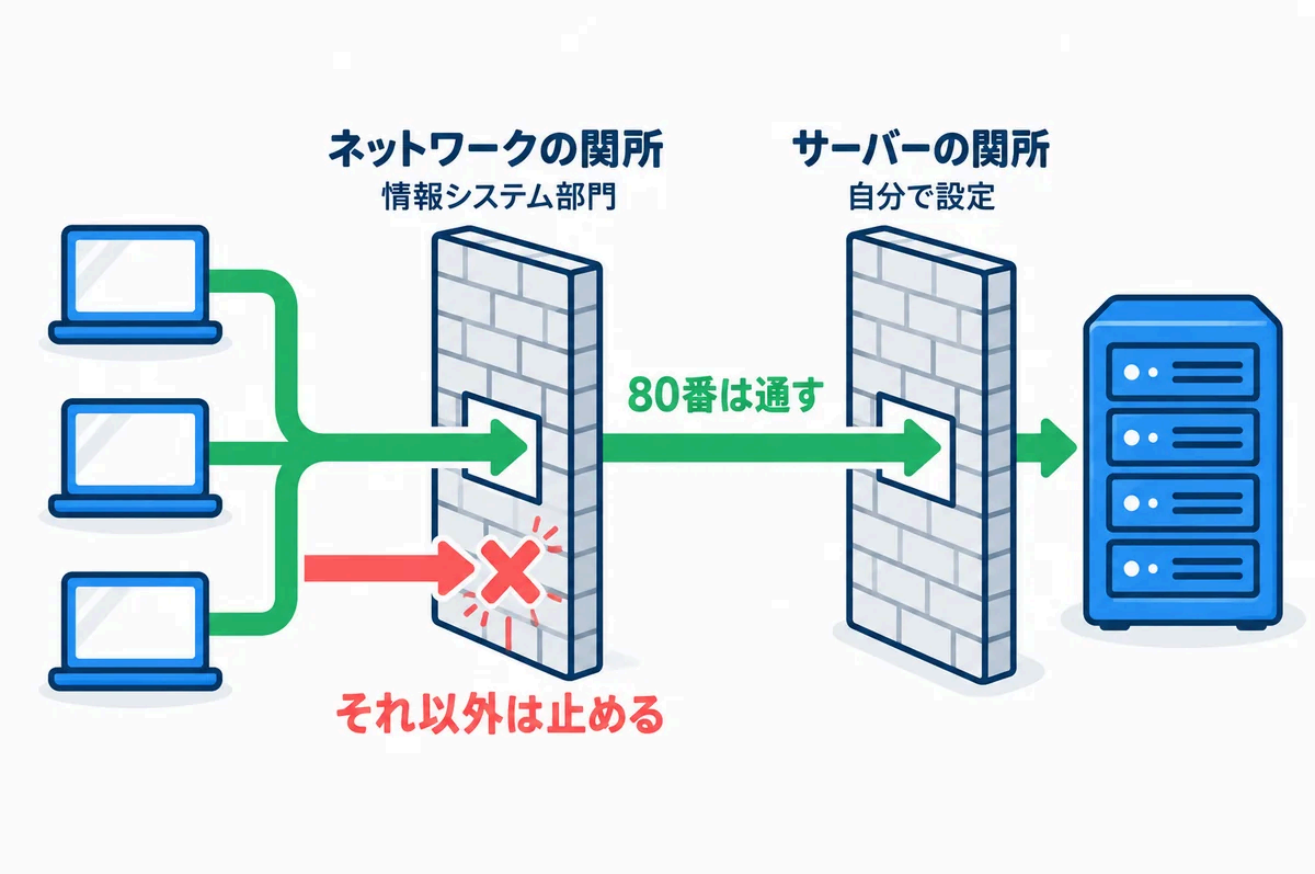


図4-3: 関所は2か所。両方で通さないと届かない

- **受信（インバウンド）**：外からサーバーに入ってくる通信。今回は「社内からの 80」と「管理用の 3389（管理者のパソコンからだけ）」だけを通します。
- **送信（アウトバウンド）**：サーバーから外へ出ていく通信。今回は「Microsoft 365 への 25」が要ります。
- 関所は1か所とは限りません。**サーバーの中の関所**（Windows Defender ファイアウォール。自分で設定できる）と、**ネットワークの途中の関所**（会社の機器。情報システム部門が設定する）があり、**両方で通っていないと届きません**。

> **用語:** ファイアウォール（firewall：決めた通信だけを通す関所。防火壁の意味）

4.4 通信を確かめる道具

やりたいこと	PowerShell のコマンド
自分の IP アドレスを見る	`Get-NetIPConfiguration` または `ipconfig /all`
名前から IP アドレスを調べる	`Resolve-DnsName webapp.example.local`
相手のポートにつながるか確かめる	`Test-NetConnection 相手の名前 -Port 80`
この機械で待ち受けているポートを見る	`Get-NetTCPConnection -State Listen`

使い方の例です。どれも、ふつうの（管理者でない）PowerShell で動きます。

コピー

```
# ① 自分の IP アドレス・サブネット・ゲートウェイ・DNS サーバーを見る
Get-NetIPConfiguration

# ② 名前を引いて、どの IP アドレスが返ってくるかを見る
Resolve-DnsName webapp.example.local

# ③ サーバーの 80 番につながるかを確かめる（社員のパソコンから打つ）
Test-NetConnection webapp.example.local -Port 80

# ④ サーバーの中で、どのポートで待ち受けているかを見る（サーバーで打つ）
#    LocalAddress が 127.0.0.1 なら「この機械の中だけ」、0.0.0.0 や :: なら「外からも」
Get-NetTCPConnection -State Listen | Sort-Object LocalPort |
    Select-Object LocalAddress, LocalPort, OwningProcess
```

コマンド	確かめること
①	`IPv4Address` が、情報システム部門から知らされた番号であること
②	`IPAddress` に、サーバーの IP アドレスが出ること。エラーなら、DNS に名前が登録されていない
③	`TcpTestSucceeded : True` なら「つながった」。`False` なら、途中の関所（ファイアウォール）で止められているか、相手が待ち受けていない
④	80（IIS）・5432（PostgreSQL）・8080／8081（アプリ）が並んでいること。5432・8080・8081 の `LocalAddress` が `127.0.0.1` であること

> **用語:** ping（ピング：相手が生きているかを確かめる、昔からある道具。会社のネットワークでは ping を止めていることが多く、「ping が通らない＝つながらない」とは限らない。Web がつながるかは、`Test-NetConnection -Port 80` で確かめる）

4.5 社内の住所の決まり（プライベート IP アドレスとサブネット）

- IP アドレスには、**インターネットで使う番号（グローバル IP アドレス）**と、**会社や家の中だけで使う番号（プライベート IP アドレス）**があります。社内のサーバーは、ふつう次の範囲のプライベート IP アドレスです。

範囲	よく使われる場所
`10.0.0.0` ～ `10.255.255.255`	大きな会社のネットワーク
`172.16.0.0` ～ `172.31.255.255`	会社のネットワーク
`192.168.0.0` ～ `192.168.255.255`	家庭や小さな事務所

- **サブネット**は、「同じ町内」の範囲です。`10.20.30.0/24` と書いてあれば、「`10.20.30.` までが同じで、最後の数（0〜255）だけが違う機械は、同じ町内」という意味です。`/24` は、サブネットマスク `255.255.255.0` と同じ意味です。
- 同じ町内の機械どうしは、直接話せます。**ほかの町内（別のサブネット）へは、出口の機械（デフォルトゲートウェイ）を通って**行きます。社員のパソコンとサーバーは、ふつう別のサブネットにあるので、間にゲートウェイと、ネットワークの関所があります。

> **用語:** プライベート IP アドレス（会社や家の中だけで使う IP アドレス。インターネットからは直接届かない）

> **用語:** サブネット（同じ範囲の IP アドレスを持つ機械のまとまり。「同じ町内」）

4.6 通信の層（だれが、どこまで受け持つか）

ネットワークの通信は、何段かの**層**に分かれていて、それぞれが別の仕事を受け持ちます。トラブルのときに「どの層で止まっているか」を考えると、原因を絞り込めます（06 トラブルシューティング）。

層	受け持つこと	今回の例	止まると
アプリケーション層	何を話すかの決まり	HTTP・SMTP・RDP・PostgreSQL の会話	「つながるのに、エラーの画面が出る」
トランスポート層	どのプログラム宛てか（ポート）と、確実に届けること	TCP の 80・25・3389	「名前は引けるのに、ポートにつながらない」
インターネット層	どの機械宛てか（IP アドレス）と、道順	`10.20.30.40`	「ほかのサブネットに届かない」
ネットワーク インターフェイス層	ケーブルや仮想のネットワークでの、隣の機械までの受け渡し	仮想マシンの NIC	「何もつながらない」

上から順に「名前は引けるか（DNS）→ ポートにつながるか（TCP）→ 正しい返事が来るか（HTTP）」と確かめていくのが、調べ方の基本です。

5. Web アプリの仕組み

5.1 ブラウザとサーバーの会話（HTTP）

1. 社員がブラウザで URL（例：`http://webapp.example.local/requests`）を開きます。
2. ブラウザは、サーバーに**リクエスト**（お願い）を送ります。「`/requests` のページをください」。
3. サーバーは、データベースから依頼を読み、ページ（HTML）を作って、**レスポンス**（返事）として返します。
4. ブラウザが、そのページを画面に表示します。



図5-1: HTTP は、リクエストとレスポンスの組で会話する

返事には、**ステータスコード**という番号が付いています。

番号	意味	よく出る場面
200	うまくいった	ふつうにページが開いた
302・303	別のページへ移って	ログインしたあと、メニューへ
403	権限が無い・断った	設定の画面を、権限の無い社員が開いた
404	見つからない	URL の打ち間違い
500	サーバーの中でエラー	アプリの不具合・データベースにつながらない
502	受付の先（アプリ）から、まともな返事が来ない	アプリが止まっている（IIS の前で起きる）

> **用語:** HTTP（エイチティーティーピー：ブラウザとサーバーの会話の決まり。リクエストとレスポンスでできている）

> **用語:** URL（ユーアールエル：Web のページの住所。`http://` （決まり） ＋ `webapp.example.local` （サーバーの名前） ＋ `/requests` （ページの場所））

> **用語:** HTML（エイチティーエムエル：Web のページの中身を書く言葉）

5.2 Web サーバー・アプリ・リバースプロキシ

今回の仕組みは、**受付**と**店員**に分かれています。

- **受付（Web サーバー。今回は IIS）**：社員のお願いを最初に受け、名前（Host）を見て本番と検証に振り分け、大きすぎる送信を断り、来たお願いを記録（アクセスログ）してから、店員に渡します。
- **店員（アプリ）**：ログインの確認、入力のチェック、データベースの読み書き、メール送信など、業務の仕事をします。
- 受付が、来たお願いを奥の店員にそのまま取り次ぐ仕組みを、**リバースプロキシ**と呼びます。IIS では、**URL Rewrite** と **ARR** という追加の部品を入れて、リバースプロキシにします。

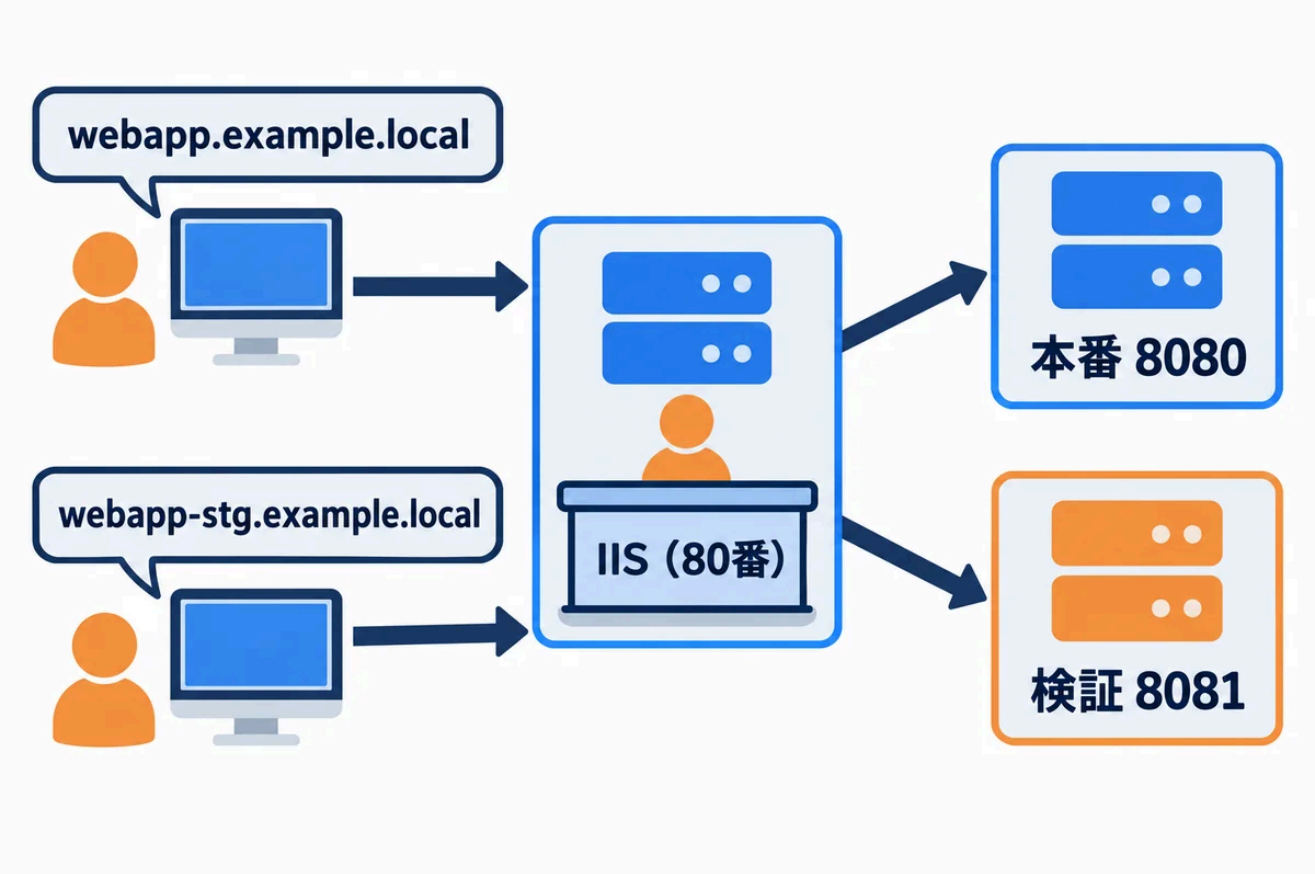


図5-2: 受付（IIS）が、来た名前を見て本番と検証に振り分ける

なぜ分けるのか:

- 1つの 80 番で、本番と検証の2つのアプリを出し分けられる。
- アプリは `127.0.0.1` だけで待ち受けられるので、外から直接さわられない。
- Windows の標準の仕組み（IIS）なので、情報システム部門も管理しやすい。あとで HTTPS にするときも、受付の設定を変えるだけで済む（参考資料）。

> **用語:** IIS（アイアイエス。Internet Information Services：Windows Server に付いている Web サーバー）

> **用語:** リバースプロキシ（reverse proxy：外からのお願いを受けて、奥のアプリに取り次ぎ、返事を外へ返す係）

> **用語:** 静的ファイル（中身が決まっていて、そのまま返せばよいファイル。CSS・JavaScript・画像など）

5.3 ログインとセッション

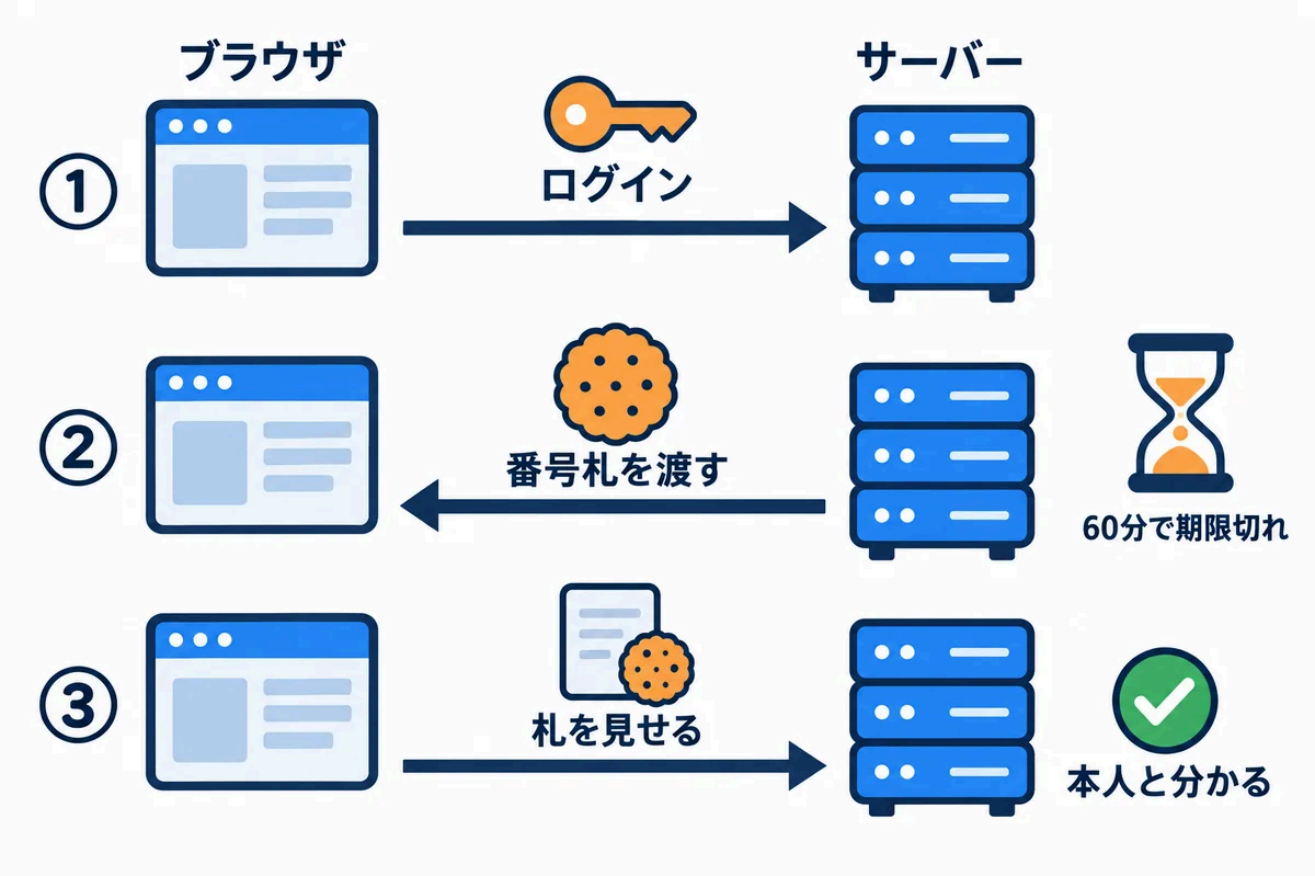


図5-3: ログインの印（セッション）を、クッキーで運ぶ

- ログインすると、サーバーは「この人はログイン済み」という印（**セッション**）を作り、その番号を、ブラウザの**クッキー**に預けます。次からのお願いには、ブラウザがクッキーを付けて送るので、サーバーは誰からのお願いかが分かります。
- 例のアプリでは、60 分操作が無いと自動でログアウトします（セッションの期限）。
- クッキーには「HTTPS のときだけ送る」（`Secure`）という印を付けられます。**HTTP で動かすときは、この印を付けない**ように、アプリの設定を変えます（6 章、手順書）。

> **用語:** クッキー（cookie：サーバーがブラウザに預ける、小さな文字のメモ。次のお願いのときに、ブラウザが自動で付けて送る）

> **用語:** セッション（session：「この人はログイン済み」という印。サーバーが番号を作り、クッキーで運ぶ）

5.4 HTTP の中身をのぞいてみる

HTTP の会話は、ただの文字です。次は、社員が依頼の一覧を開いたときの**リクエスト**の例です（説明のために短くしてあります）。

コピー

```
GET /requests HTTP/1.1
Host: webapp.example.local
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) ...
Accept: text/html
Cookie: session=AbC123...
```

行	意味
`GET /requests HTTP/1.1`	「`/requests` のページをください」。`GET` は「見せて」、`POST` は「これを送るので処理して」（保存・ログインなど）
`Host:`	どの名前のサイト宛てか（4.2）。IIS は、ここを見て本番と検証に振り分ける
`User-Agent:`	どのブラウザか
`Cookie:`	前にサーバーから預かったクッキー（5.3）。ログイン中かどうかが、これで分かる

サーバーの**レスポンス**の例です。

コピー

```
HTTP/1.1 200 OK
Content-Type: text/html; charset=utf-8
Content-Length: 5120

<!doctype html>
<html lang="ja"> ...（ページの中身）
```

1行目の `200 OK` がステータスコード（5.1 の表）、空の行より上が**ヘッダー**（付け札）、下が**ボディ**（中身）です。

ブラウザで本物を見る方法: Edge か Chrome で `F12` キーを押し、「ネットワーク」（Network）のタブを開いてから、ページを読み直します。並んだ行の1つをクリックすると、本物のリクエストとレスポンスのヘッダーを見られます。トラブルのときに、ステータスコードやクッキーを確かめるのに使います（06 トラブルシューティング）。

PowerShell からも、HTTP のお願いを出せます。

コピー

```
# サーバーの /health にお願いを出して、ステータスコードを表示する
(Invoke-WebRequest http://webapp.example.local/health -UseBasicParsing).StatusCode
```

確かめること: `200` が表示されること。エラーになったら、その文に出ているステータスコード（404・502 など）を、5.1 の表と照らし合わせます。

> **用語:** ヘッダー（リクエストやレスポンスに付く、付け札の情報。`Host`・`Cookie`・`Content-Type` など）

> **用語:** GET・POST（HTTP のお願いの種類。GET は「見せて」、POST は「これを送るので処理して」）

> **用語:** 開発者ツール（ブラウザで `F12` を押すと開く道具。通信の中身やエラーを見られる）

6. HTTP と HTTPS（今回は HTTP）

6.1 違い

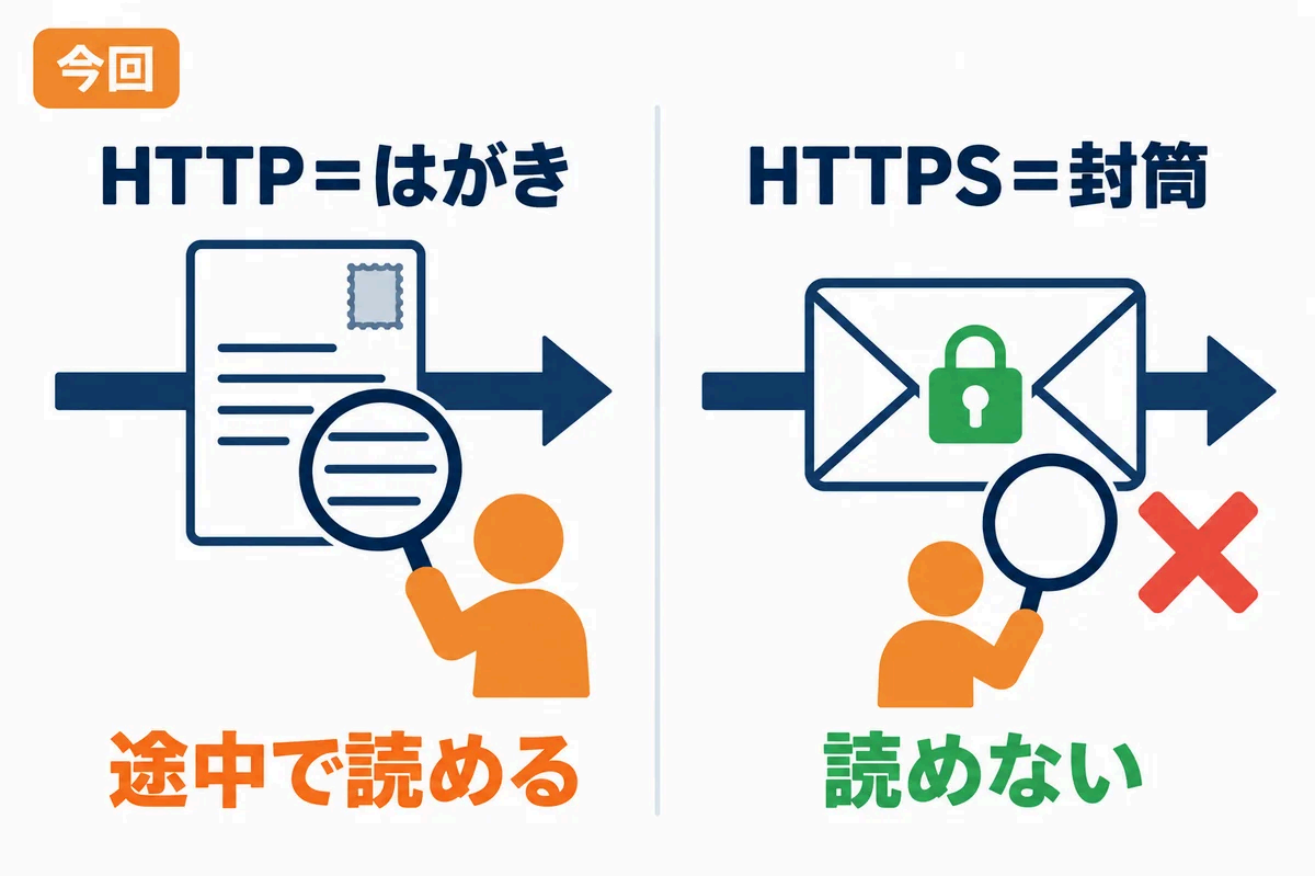


図6-1: HTTP ははがき、HTTPS は封をした封筒

- **HTTP** は、ブラウザとサーバーの会話を、**そのままの文字**で送ります。途中の機械で、パスワードや入力した内容を読むことができます。
- **HTTPS** は、同じ会話を**暗号化**して送ります。暗号化には、サーバーの身分証明書（**証明書**）が要ります。

	HTTP（今回）	**HTTPS（参考資料）**
中身	そのままの文字	暗号化される
ポート	80	443
準備	要らない	証明書を用意し、受付（IIS）に入れる
ブラウザの表示	アドレス欄に「保護されていない通信」などと出る	鍵のマーク
`Secure` の付いたクッキー	**受け取られない**（アプリの設定を変える必要がある）	使える

6.2 今回 HTTP にする理由と、気を付けること

今回の会社の環境では、社内で HTTPS を使えない（証明書を用意できない）ため、**HTTP で動かします**。HTTP にするときは、次のことに気を付けます。

- **社内のネットワークからだけ**使えるようにする（ファイアウォールで、社外から届かないようにする）。
- 社員が、ほかのサービスと**同じパスワードを使い回さない**ように案内する（途中で読まれても、ほかに広がらないように）。
- アプリの部品の中には、HTTPS を前提にしている物がある（例：`Secure` の付いたクッキー）。手順書で、HTTP 用の設定に変える所を説明する。
- 将来、証明書を用意できるようになったら、HTTPS に切りかえる。受付（IIS）とアプリの設定を少し変えるだけで済むように、手順書を作ってある。

HTTPS と証明書の考え方、切りかえの手順は、**参考資料「HTTPS にする場合」**に分けて書いています。

7. データベースの基礎

7.1 データベースとは

データベース（DB）は、データを**表の形**で整理して保管する、帳簿の棚です。依頼・図面・社員データ・承認の記録などを保管します。

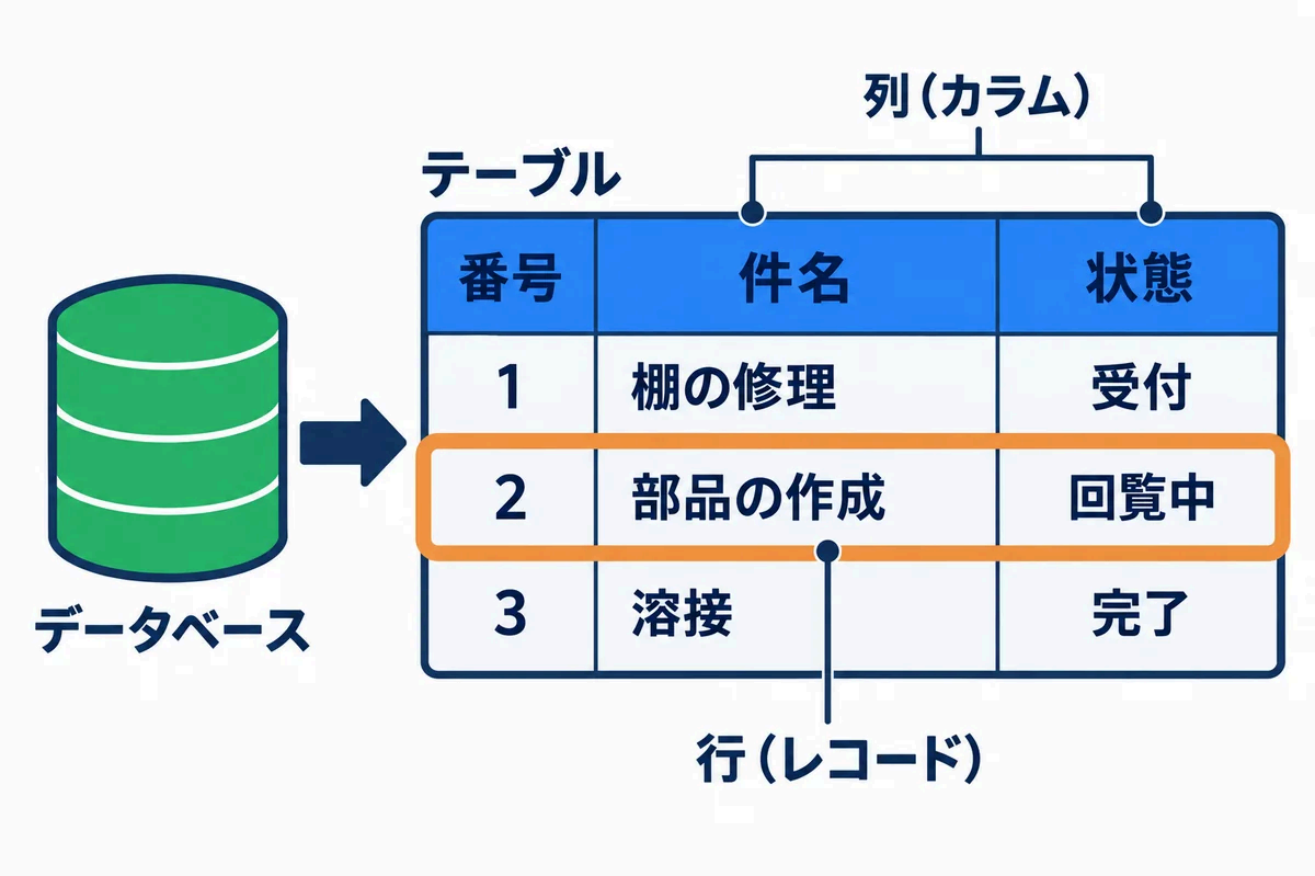


図7-1: テーブルは、行（1件分）と列（項目）でできた表

- 1つの表を**テーブル**、表の1行を**レコード（行）**、表の列を**カラム（列）**と呼びます。例：「依頼」テーブルの1行が、依頼1件です。
- データベースに「この条件の依頼を出して」とお願いする言葉が **SQL** です。
- データベースを動かすソフトを **DBMS**（データベース管理システム）と呼び、今回は **PostgreSQL**（ポストグレス）を、**仮想マシンの中に**入れて使います。

> **用語:** SQL（エスキューエル：データベースに、データの出し入れをお願いする言葉。例： `SELECT * FROM requests WHERE status = '回覧中';` ）

> **用語:** PostgreSQL（ポストグレスキューエル：無料で使える、広く使われているデータベース。Windows にも入れられる）

SQL の例を1つだけ見ておきます。「状態が『回覧中』の依頼を、新しい順に 10 件出して」というお願いです。

コピー

```
SELECT id, title, status, created_at   -- 出してほしい列
FROM requests                          -- どのテーブルから
WHERE status = '回覧中'                 -- どの行を（条件）
ORDER BY created_at DESC               -- 並べ方（DESC は新しい順）
LIMIT 10;                              -- 最大の件数
```

SQL の書き方と、データベースへのログインの仕方は、**02 SQL と PostgreSQL の使い方**で、サンプルを動かしながら説明します。

7.2 データベースのユーザーと接続

- データベースにも、**ユーザー**（ログインする名前）と**権限**があります。アプリは、アプリ専用のユーザーでつなぎます。そのユーザーは、**自分のデータベースだけ**を読み書きできるようにします。
- アプリがデータベースにつなぐための情報（場所・ポート・データベース名・ユーザー名・パスワード）をまとめたものを、**接続文字列**と呼びます。例：`postgres://webapp_prod_app:パスワード@127.0.0.1:5432/webapp_prod`。パスワードが入っているので、秘密の情報として扱います。

7.3 トランザクションとマイグレーション



図7-2: トランザクションは「全部やるか、全部やめるか」

- **トランザクション**は、「全部うまくいくか、全部やめるか」をまとめる仕組みです。承認したときに「承認の記録を残す」と「次の人へ回す」の片方だけが起きないように使います（アプリが受け持つ）。
- **マイグレーション**は、テーブルの形（列の追加など）を、番号付きの手順として管理し、順に当てていく仕組みです。アプリを新しい版にするとき、テーブルの形も一緒に変えます（手順書の「更新」の章）。

7.4 バックアップとリストア

- **バックアップ**は、データの控えです。PostgreSQL では、`pg_dump`（ピージーダンプ）という道具で、データベースの中身を1つのファイルに書き出せます。
- **リストア**は、控えから元に戻すことです。**バックアップは、実際にリストアできることを確かめて、初めて意味があります**（9章）。

7.5 SQL Server を指定されたら

会社によっては、データベースに Microsoft の **SQL Server** を使うよう決まっていることがあります。考え方（テーブル・SQL・ユーザー・バックアップ）は同じですが、次の点が違います。

	PostgreSQL（この資料）	SQL Server
費用	無料	Express 版は無料（1つのデータベースが 50GB まで（2025 の版。2022 までは 10GB）、CPU は 4 コアまで、などの制限）。それ以上は有料のライセンス
管理の道具	psql・pgAdmin	SQL Server Management Studio（SSMS）
バックアップ	`pg_dump`・`pg_restore`	`BACKUP DATABASE`・`RESTORE DATABASE`（SSMS からも）
アプリの部品	Rust: sqlx・tokio-postgres など／Django: 標準で対応	Rust: tiberius など／Django: mssql-django（Microsoft が出している部品）

どちらにするかは、アプリを作り始める前に決めます。途中で変えると、アプリの部品と SQL の書き方の一部を直すことになります。

8. メール送信の基礎

8.1 SMTP とメールの配達

- メールを送るときの決まりが **SMTP** です。アプリは「宛先・件名・本文」を SMTP で**メールサーバー**に渡し、メールサーバーが相手に配達します。
- 会社のメールが **Microsoft 365**（**Exchange Online**） なら、アプリは、Microsoft 365 にメールを渡して、配達してもらいます。

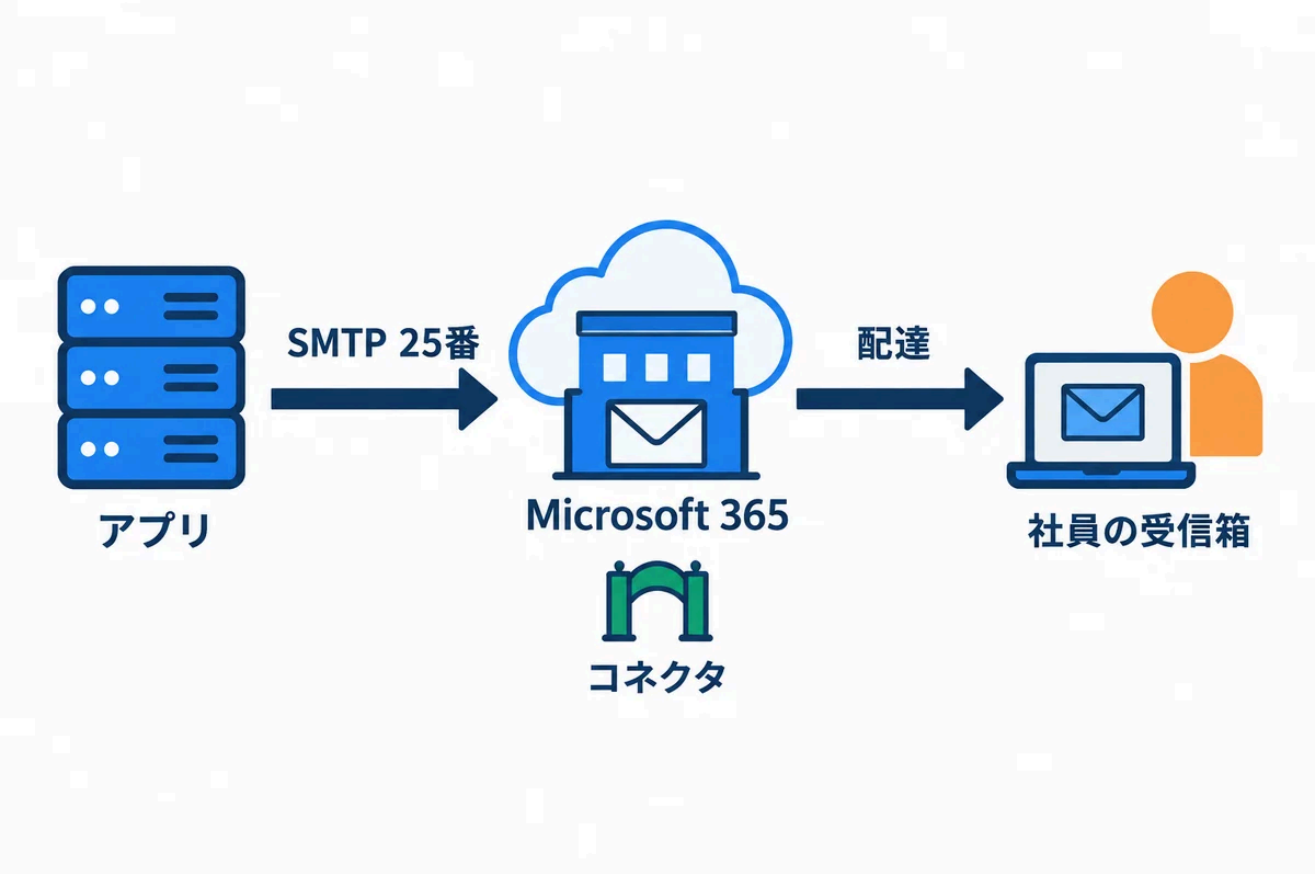


図8-1: アプリは Microsoft 365 にメールを渡し、配達してもらう

8.2 SMTP リレーとコネクタ

Microsoft 365 にメールを渡す方法はいくつかありますが、この資料では **SMTP リレー**を使います。アプリにパスワードや鍵を持たせなくてよいからです。Microsoft の公式のドキュメント（2026-10 時点）によると、条件は次のとおりです。

- Microsoft 365 の管理画面に、**コネクタ**（「この送り主からのメールは、会社のものとして受け付ける」という設定）を作る。送り主は、**会社のインターネットの出口の、固定の公開 IP アドレス**（ほかの会社と共有していないもの）か、証明書で見分ける。
- アプリは、会社のドメインの **MX エンドポイント**（例：`example-com.mail.protection.outlook.com`）の **25 番ポート**に、SMTP でつなぐ。
- 送り主のアドレスは、会社のドメインのアドレスなら、メールボックスが無くてもよい（例：`noreply@example.com`）。
- 迷惑メールと判定されないように、ドメインの **SPF** レコードに、その IP アドレスを書き足す。

コネクタ・SPF・出口のファイアウォールは、**情報システム部門の作業**です。依頼する内容は、03 付録の依頼シートにまとめます。

> **用語:** SMTP（エスエムティーピー：メールを送る・配達するときの決まり。ふつう 25 番ポートを使う）

> **用語:** SMTP リレー（ほかのメールサーバーに、メールの配達を任せること。今回は Microsoft 365 に任せる）

> **用語:** MX エンドポイント（そのドメイン宛てのメールを受け付ける、Microsoft 365 の受付の住所）

> **用語:** SPF（エスピーエフ：「このドメインのメールは、この IP アドレスから送られます」と DNS に書いておく仕組み。書いていない所から送ると、なりすましと疑われる）

9. 運用の基礎

建てて終わりではありません。**毎日、安心して使える状態を保つ**ことを、運用と呼びます。

9.1 ログ

- **ログ**は、プログラムが書く「何が起きたか」の記録です。エラーが起きたとき、最初に見るのがログです。
- ログは、どんどん大きくなるので、**古いログを消すか、別のファイルに分ける**（ローテーション）仕組みが要ります。ディスクがいっぱいになると、アプリもデータベースも止まります。
- Windows 自身のログは、**イベント ビューアー**（`eventvwr.msc`）で見られます。

9.2 バックアップ

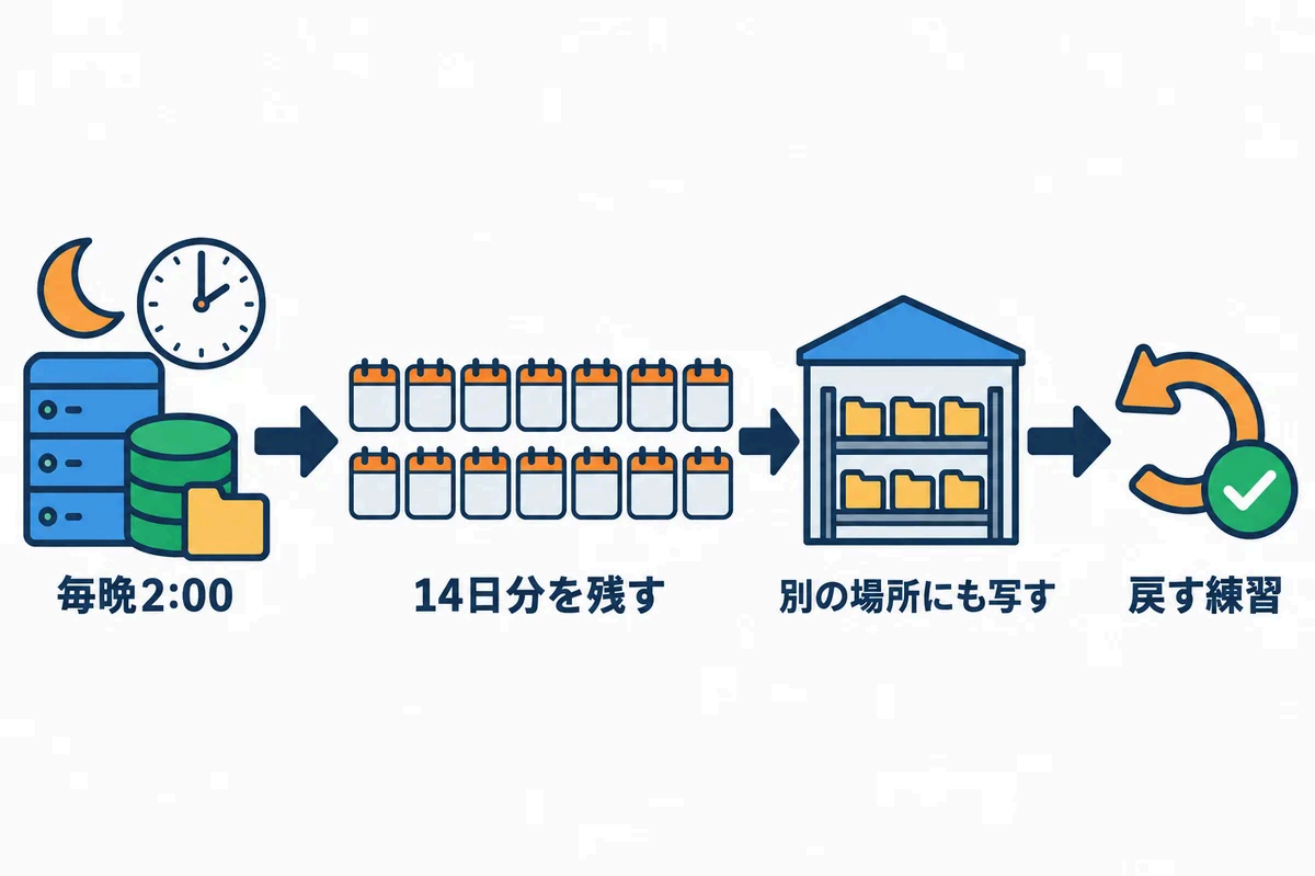


図9-1: 毎晩の控え。14 日分を残し、別の場所にも写し、戻す練習をする

- 例のアプリでは、**毎晩、データベースと添付ファイルのフォルダをバックアップし、14 日分を残します。**
- バックアップを**同じサーバーの中だけ**に置くと、サーバーが壊れたときに一緒に失います。**別の場所**（会社のファイルサーバーなど）にも写しを置きます。
- 仮想マシンのスナップショットは、バックアップの代わりになりません。スナップショットは、同じ場所に置かれ、長く残すと遅くなるからです。
- **年に何回か、バックアップから実際に戻す練習（リストアの訓練）**をします。

9.3 定期処理

- 毎日決まった時刻に動かす仕事を、**定期処理**と呼びます。例のアプリでは、毎朝 8:00 の期限・承認のチェックのメールと、夜間のバックアップです。
- Windows では、**タスク スケジューラ**で「毎日何時にこのコマンドを動かす」と決められます。

9.4 本番と検証

- **本番環境**は、社員が実際に使う環境です。**検証環境**は、新しい版や設定を、先に試す環境です。
- 今回は、同じサーバーに本番と検証を同居させ、**フォルダ・ポート・データベースを分けます。**社員が開く名前も分けます（4.2 の Host ヘッダー）。検証で確かめてから、本番に反映します。

9.5 デプロイと切り戻し

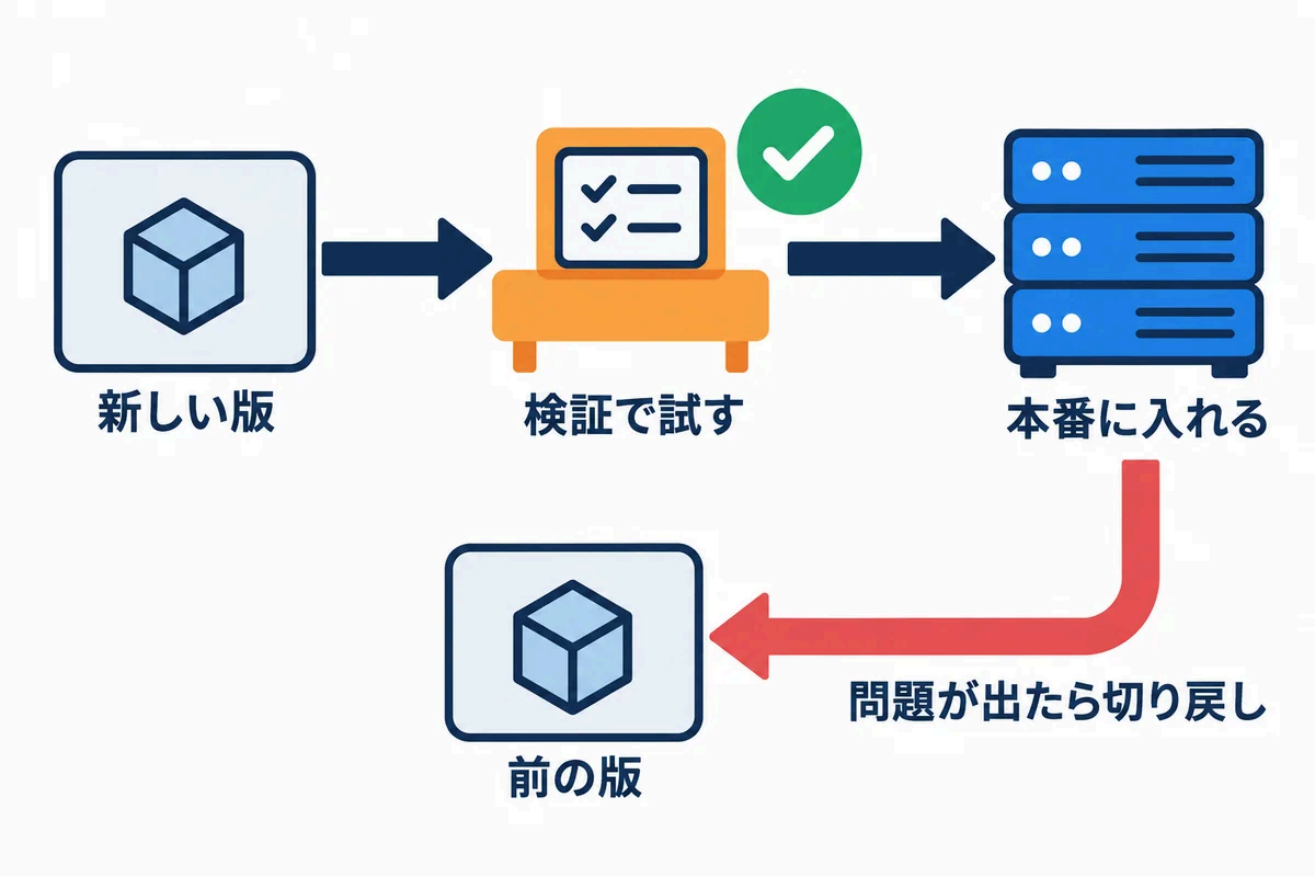


図9-2: 検証で試してから本番へ。だめなら前の版に戻す

- 新しい版のアプリをサーバーに入れることを、**デプロイ**と呼びます。
- 新しい版に問題があったとき、前の版に戻すことを、**切り戻し**（ロールバック）と呼びます。**切り戻しの手順を、デプロイの前に決めておく**のが大事です。

9.6 期限のある物

物	期限が切れると	対策
OS・ソフトのサポート	安全の修正が出なくなる	Windows Server・PostgreSQL・アプリの部品のサポート期限を表にしておく
パスワード	会社の決まりで、管理者のパスワードに期限があることがある	サービス用アカウントのパスワードの期限が切れると、サービスが起動しなくなる。手順書では、パスワードの無いアカウント（仮想アカウント）を使う
（HTTPS にしたら）証明書	全員の画面に警告が出る	参考資料

10. セキュリティの基礎

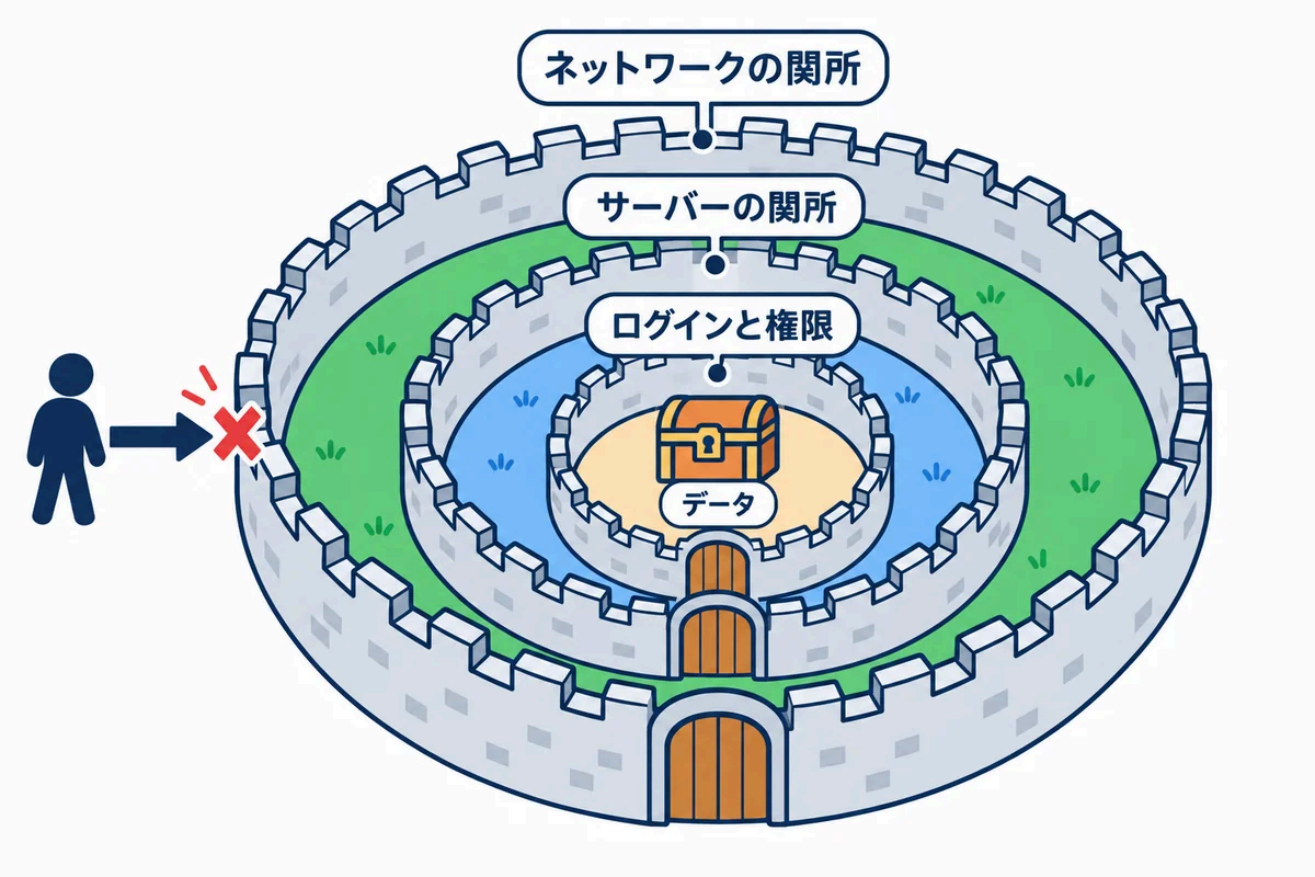


図10-1: 多層防御。1つ破られても、次の壁で止める

考え方	今回の具体例
最小権限: 必要な権限だけを渡す	アプリはサービス用アカウントで動かす。DB のユーザーは自分のデータベースだけ
入口を絞る: 外に見せる物を減らす	外から届くのは IIS の 80 と、管理用の 3389 だけ。アプリと DB は 127.0.0.1
多層防御: 1つ破られても次で止める	サーバーのファイアウォール＋ネットワークの関所＋アプリのログイン
秘密を守る: パスワードや鍵を見せない	設定ファイルは管理者とアプリ用アカウントだけが読める。Git やメールに貼らない
通信を守る: 途中で読まれないように	今回は HTTPS を使えないので、社内だけに絞り、パスワードの使い回しをしないよう案内する。使えるようになったら HTTPS にする（参考資料）
更新する: 弱点を直し続ける	Windows Update、PostgreSQL・IIS の部品・アプリの部品の更新を、検証環境で確かめてから入れる
記録する: 後から追えるように	アプリの操作の記録、IIS のアクセスログ、Windows のイベントログを残す

> **用語:** 最小権限（その仕事に必要な権限だけを渡すこと。乗っ取られても、被害を小さくできる）

用語の索引

用語	短い説明	章
Active Directory（AD）	会社のパソコンとユーザーをまとめて管理する仕組み	3
ARR・URL Rewrite	IIS をリバースプロキシにするための追加の部品	5
GET・POST	HTTP のお願いの種類（見せて／処理して）	5
DNS	名前と IP アドレスを結び付ける電話帳	4
FQDN	ドメイン名まで付けた完全な名前	4
Host ヘッダー	お願いに付く「どの名前のサイト宛てか」の情報	4
HTTP・HTTPS	ブラウザとサーバーの会話の決まり／それを暗号化したもの	5・6
IIS	Windows Server の Web サーバー	5
IP アドレス	ネットワーク上の番号の住所	4
localhost（127.0.0.1）	この機械自身を表す住所	4
MX エンドポイント	ドメイン宛てのメールを受け付ける Microsoft 365 の住所	8
PostgreSQL	今回使うデータベース	7
PowerShell	Windows を文字のコマンドで操作する道具	3
SMTP・SMTP リレー	メールを送る決まり／配達を任せること	8
SPF	ドメインのメールを送ってよい IP を DNS に書く仕組み	8
SQL	データベースにお願いする言葉	7
vCPU	仮想マシンに割り当てられた CPU の数	2
インフラ	アプリを動かす土台の全部	はじめ
オンプレミス	会社の設備の上でシステムを動かすこと	はじめ・2
仮想マシン（VM）	1台の機械の中に作った仮のコンピューター	2
クッキー・セッション	ブラウザに預けるメモ／ログイン状態の印	5
検証環境・本番環境	試す環境／実際に使う環境	9
最小権限	必要な権限だけを渡すこと	3・10
サービス	人がいなくても OS が動かし続けるプログラム	3
スナップショット	仮想マシンのその瞬間の状態の保存	2・9

用語	短い説明	章
タスク スケジューラ	決まった時刻にコマンドを動かす Windows の機能	9
データベース・テーブル	データを表で保管する棚／その1つの表	7
デプロイ・切り戻し	新しい版を入れること／前の版に戻すこと	9
トランザクション	全部うまくいくか、全部やめるかをまとめる仕組み	7
バックアップ・リストア	データの控え／控えから戻すこと	7・9
ファイアウォール	決めた通信だけを通す関所	4
プライベート IP アドレス	社内だけで使う IP アドレス	4
ポート番号	機械の中のどのプログラム宛てかを表す番号	4
マイグレーション	テーブルの形の変更を、番号付きの手順で当てる仕組み	7
文字コード・文字化け	文字と番号の対応の決まり／その食い違い	0

| リバースプロキシ | 外からのお願いを奥のアプリに取り次ぐ係 | 5 | | ログ | 何が起きたかの記録 | 9 |